Word Representation I: Meaning, Sets, and Retrieval

Introduction to LLMs · Word Representation, day 1 of 3

Representing Data

- Numeric Data:
  - E.g. credit score:

Monthly Income	Number Of Open Credit Lines And Loans	Number Of Times 90 Days Late	Number Real Estate Loans Or Lines	Number Of Time 60-89 Days Past Due Not Worse	Number Of Dependents
9120	13	0	6	0	2
2600	4	0	0	0	1
3042	2	1	0	0	0
3300	5	0	0	0	0
63588	7	0	1	0	0
3500	3	0	1	0	1
NA	8	0	3	0	0
3500	8	0	0	0	0

Give Me Some Credit (gmsc): https://www.kaggle.com/c/GiveMeSomeCredit

Representing Data

- Numeric Data:
  - E.g. credit score:
- Images:
  - Gray scale or RGB



MNIST dataset
Handwritten numbers

- Videos:
  - Images on a timeline

How can we represent words (tokens) with vectors?

Not as straightforward: words come with no natural coordinates.

First question: what should such a vector capture?

Representing `Textual` Data

- The vast majority of rule--based and statistical NLP work regarded words as atomic symbols.
  - Recall: grammar rules

```
Det → the | a | an
Adj → old | red | happy | …
N   → dog | park | ice-cream | contumely | run | …
V   → saw | ate | run | disdained | …
P   → in | to | on | under | with | … }
```

- This is a vector with one 1 and a lot of zeroes:

`[0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]`

  - The “one-hot” representation
  - i-th word in the dictionary:

$$v_i = 1, \forall j \neq i, v_j = 0$$

Representing `Textual` Data: Problems

- There are a lot of words!
  - Oxford English Dictionary: 500,000+ entries
  - Longman Dictionary of Contemporary English: 230,000 words
  - Brysbaert et al. (2016): 42,000 lemmas
  - As a result, a lot of BIG vectors!
- No useful similarity information:
  - Motel: `[0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]`
  - Hotel: `[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]`
  - Linguist: `[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0]`

  - cos_sim(motel, hotel)=0, cos_sim(motel, linguist)=0, cos_sim(hotel, linguist)=0

Last lecture: the Lesk algorithm

Simplified Lesk (Lesk, 1986): pick the sense whose dictionary gloss shares the most words with the context.

$$\hat{s} = \arg \max_{s \in \text{senses}(w)} \left| \text{gloss}(s) \cap \text{context}(w) \right|$$

Context: “I went to the **bank** to deposit my money.”

Sense	**Gloss (WordNet)**	**Overlap**
bank (finance)	a financial institution that accepts deposits and channels the money into lending activities	2
bank (river)	sloping land (especially the slope beside a body of water)	0

Overlap counted after stemming (deposits → deposit). Lesk, M. (1986). Automatic sense disambiguation using machine readable dictionaries. SIGDOC.

What Lesk quietly assumes

1. **The meaning of a sense is written down in other words.**

   A gloss is just a bag of words. Lesk never looks “outside” the vocabulary.

2. **Words that occur close together are related.**

   Overlap between gloss and context is taken as evidence of shared meaning.

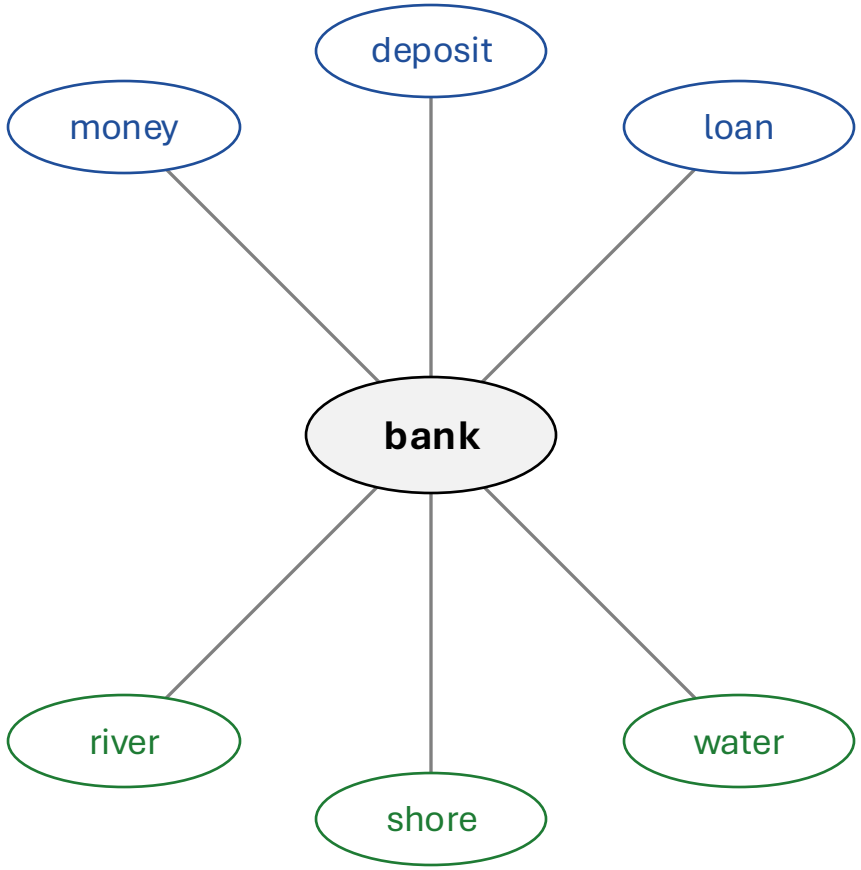
3. **The weak point: glosses are short and hand-written.**

   Overlaps are rare, and only exact matches count (deposit ≠ savings).

Today: keep assumptions 1 and 2, but replace the dictionary with a corpus.
A word’s “gloss” becomes all the contexts it occurs in.

Meaning is relational

- A dictionary defines every word using other words. Follow the definitions and you never leave the vocabulary.
- So a word's meaning is not a property of the word alone: it is its place in a web of relations to other words.
- "In language there are only differences." (F. de Saussure, 1916)
- Lesk already relied on this: a sense *is* the words of its gloss.



Two clusters of neighbours ≈ two senses

From meaning to corpus statistics

In an ideal corpus, words that occur in close contexts are likely to be related.

“Ideal” = large, natural, and representative of how words are actually used. Every method below then has to make three decisions:

Decision	**Options we will see**
1. What counts as a context?	a document · a window of ±k words · the words before the target
2. How is co-occurrence recorded?	membership · counts · weighted counts · probabilities
3. How are two words compared?	overlap of sets · angle between vectors · conditional probability

Caveat: related ≠ similar

Word pair	**Relation**	**Shared context**
car – automobile	synonyms	“park the ___ in the garage”
hot – cold	antonyms	“the coffee is too ___”
coffee – cup	association	“a _ *of …*”, “*pour the* _”
doctor – nurse	same situation	“the ___ at the hospital”

- All four pairs share contexts, so co-occurrence alone cannot tell them apart.
- Distributional methods measure **relatedness**. Every method in this course inherits this, including LLMs.

Relatedness needs structure

Relatedness is abstract. To compute with it we impose structure on it, and each step adds more:

Structure imposed	**Context of a word**	**A word is …**	**Relatedness is …**	**Methods**
Sets	the documents it is a member of	an atomic symbol (an index)	counted shared membership	Lesk, TF-IDF, BM25
Geometry	a document or a ±k window (order ignored)	a point in $\mathbb{R}^k$	angle / distance, linear offsets	LSA, word2vec, GloVe
Positions	the ordered sequence around it	a vector that depends on its context	probability of a word at a position	neural LMs (RNN → Transformer)

Each row keeps the idea of the row above and adds structure. Today: sets. Next: geometry. Then: positions.

Structure 1: sets

- A document is a set (strictly: a multiset) of terms. A term is a **member** of the documents it occurs in.
- Counting membership gives the statistics of today: tf (how often t occurs in d) and df (in how many documents t occurs).
- Relevance of a document to a query = weighted overlap of their term sets.
- Relatedness of two words = overlap of the sets of documents they belong to.

Lesk (last lecture)

$$\text{score}(s) = \left| G_s \cap C \right|$$

TF-IDF (today)

$$\text{score}(q, d) = \sum_{t \in T_q \cap T_d} w(t, d)$$

Same structure: intersect two sets. TF-IDF and BM25 only change how each shared member is weighted.

Information retrieval with sets

- Introduction: documents, collections, relevance
- The inverted index: storing set-membership statistics
- Search and relevance scoring
- TF-IDF and BM25
  - How to evaluate a retrieval system follows next lecture, after we have more systems to compare.

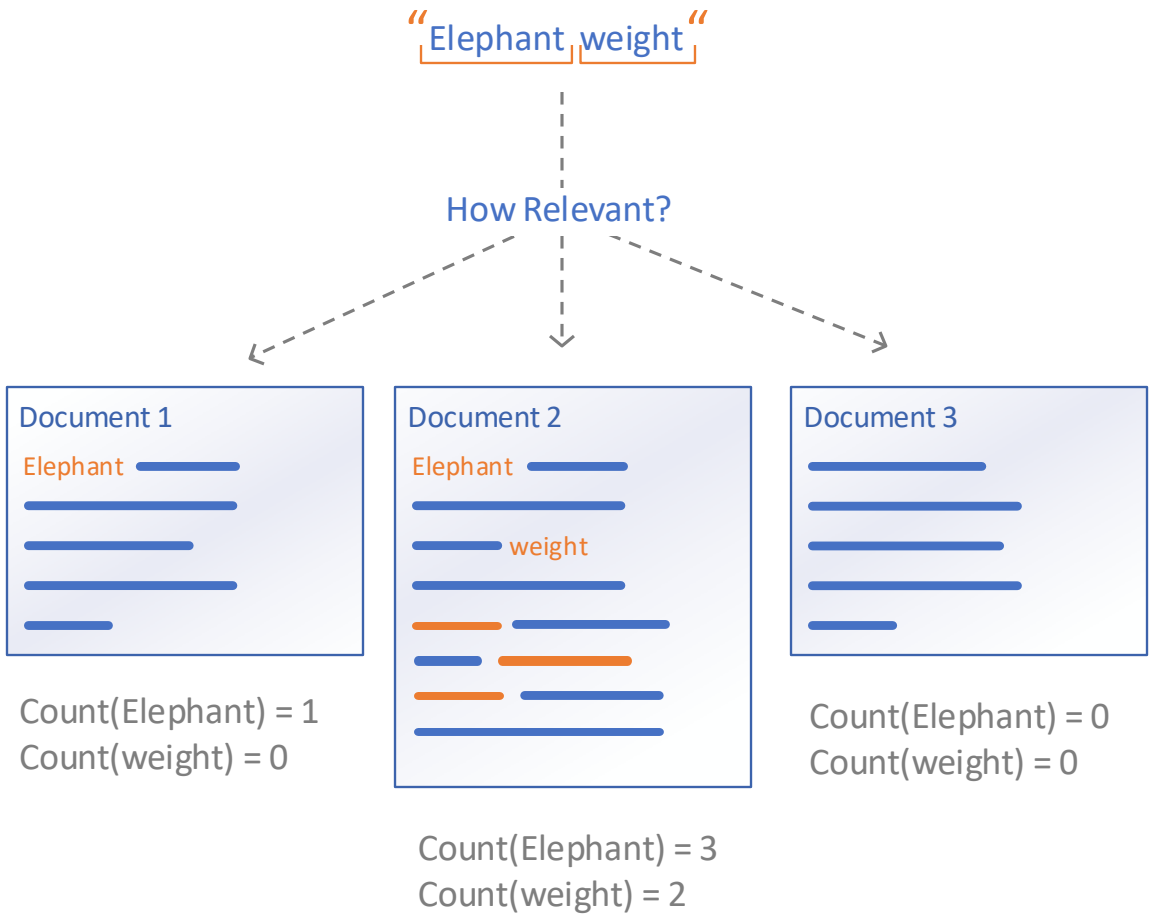
Retrieval part based on: S. Hofstätter, Introduction to Information Retrieval, Lecture 1 (Foundations of IR) and Lecture 3 (Relevance & Scoring), TU Wien. github.com/sebastian-hofstaetter/teaching

Information Retrieval

- Basically, Google Search.
- Tons of documents.
- 1 Search query.

Today's setting: rank documents for a query using only *which terms occur in which documents*.

Relevance (based on text content)



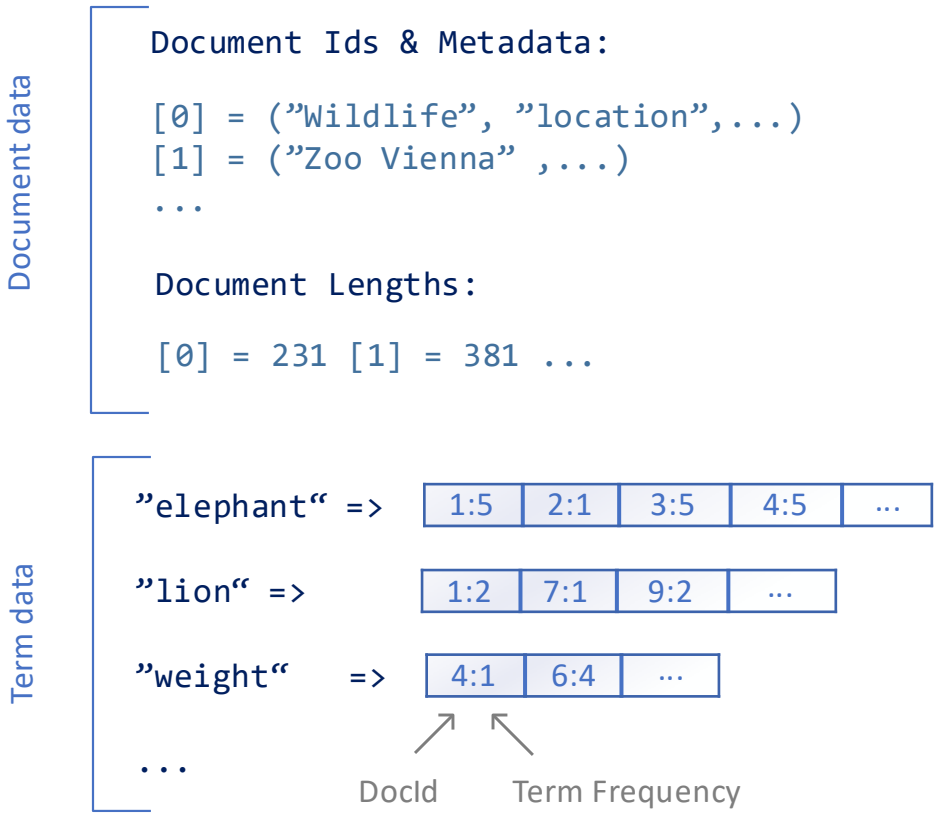
- If a word appears more often -> more relevant
- Solution: count the words
- If a document is longer, words will tend to appear more often -> take into account the document length
- Counting only when we have a query is inefficient

Inverted Index

- Inverted index allows to efficiently retrieve documents from large collections
- Inverted index stores all statistics per term (that the scoring model needs)
  - **Document frequency:** how many documents contain the term
  - **Term frequency per document:** how often does the term appear per document
  - Document length
  - Average document length

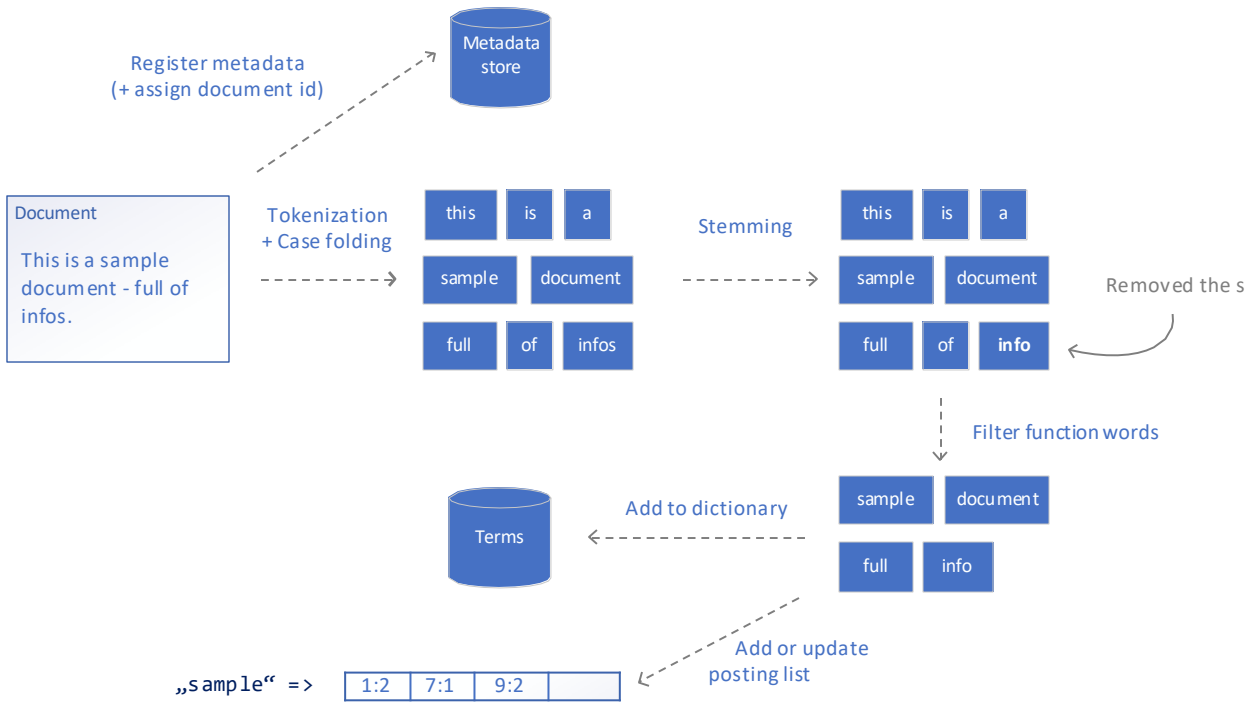
- Save statistics in a format that is accessible **by a given term**
- Save metadata of a document (Name, location of the full text, etc..)

Inverted Index



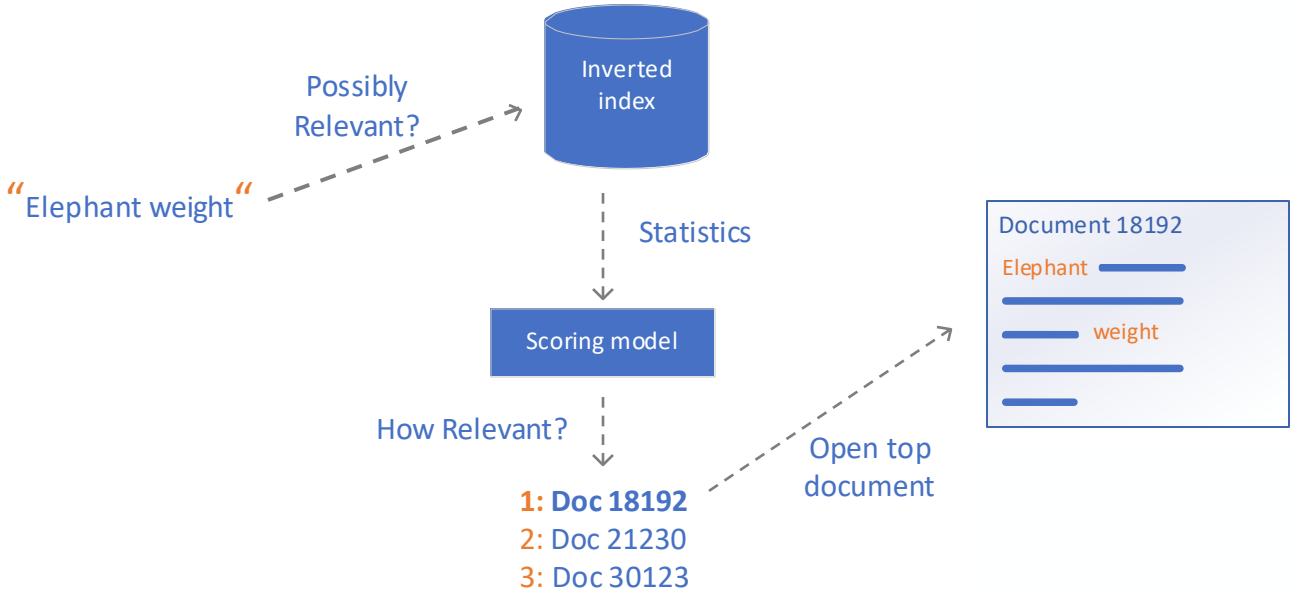
- Every document gets an internal document id
- Term dictionary is saved as a search friendly data structure (more on that later)
- Term Frequencies are stored in a “posting list” = a list of doc id, frequency pairs

Creating the Inverted Index



- Simplified example pipeline
- Linguistic models are language dependent
- A query text and a document text both have to undergo the same steps

Querying the Inverted Index



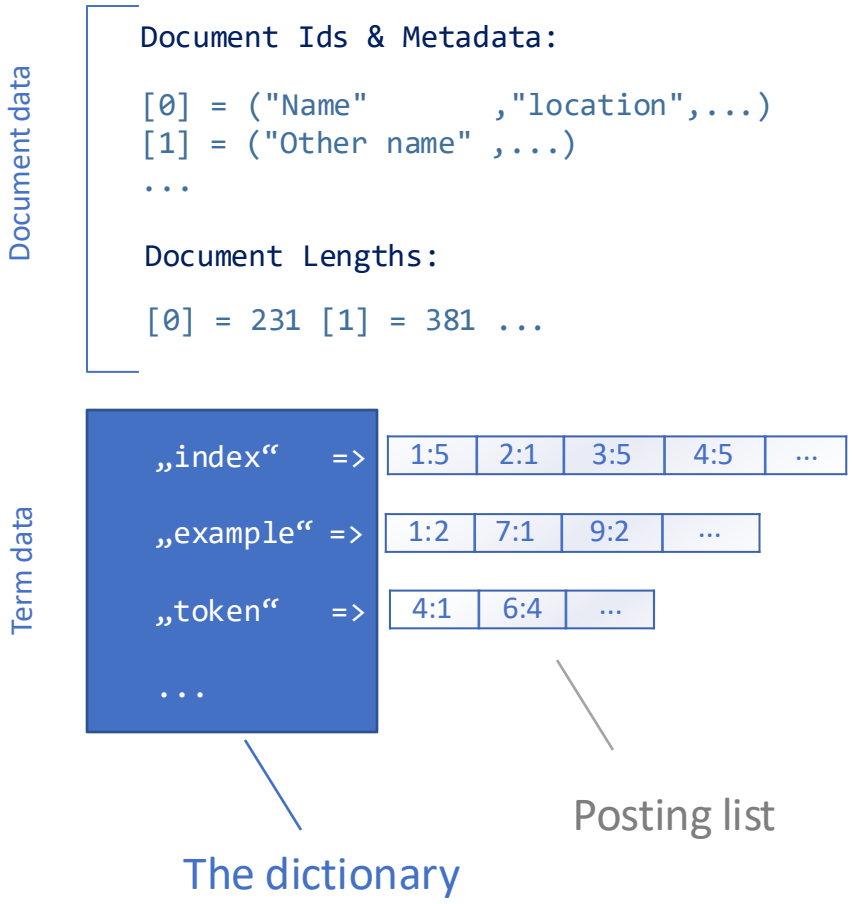
- No need to read full documents
- Only operate on frequency numbers of potentially relevant documents*
- Sort documents based on relevance score – retrieve most relevant documents

* it's not that easy because a document could be relevant without containing the exact query terms – but for now keep it simple

Types of queries (including, but not limited to)

- **Exact matching**: match full words and concatenate multiple query words with “or”
- **Boolean queries**: “and” / “or” / “not” operators between words
- **Expanded queries**: automatically incorporate synonyms and other similar or relevant words into the query
- **Wildcard queries**, **phrase queries, phonetic queries** (e.g. Soundex) …

Inverted Index: Dictionary



- Dictionary<T> maps text to T
  - T is a posting list or potentially other data about the term depending on the index
- Wanted properties:
  - Random lookup
  - Fast (creation & especially lookup)
  - Memory efficient (keep the complete dictionary in memory)
- Naturally, there are a lot of choices

Scoring model

- Input: statistics, Output: floating point value (i.e. the score)
- Evaluated pairwise – 1 query, 1 document: $score\ (q, d)$
- Capture the notion of relevance in a mathematical model

Today we focus on free-text queries & „ad-hoc“ document retrieval

(document content only)

Search algorithm

float *Scores*={}

for each query term *q*

fetch posting list for *q*

for each pair(*d*, $tf_{t,d}$) in posting list

if *d* not in *Scores* **do** *Scores*[*d*]=0

Scores[*d*] += score(q, d, $tf_{t,d}$, …)

return Top *K* entries of *Scores*

We transform information back to a document centric view (from the term centric view in the inverted index)

Relevance limitations

- "Relevance" means relevance to the need rather than to the query
  - "Query" is shorthand for an instance of information need, its initial verbalized presentation by the user
- Relevance is assumed to be a binary attribute
  - A document is either relevant to a query/need or it is not
- We need these simplifications to create & evaluate mathematical models

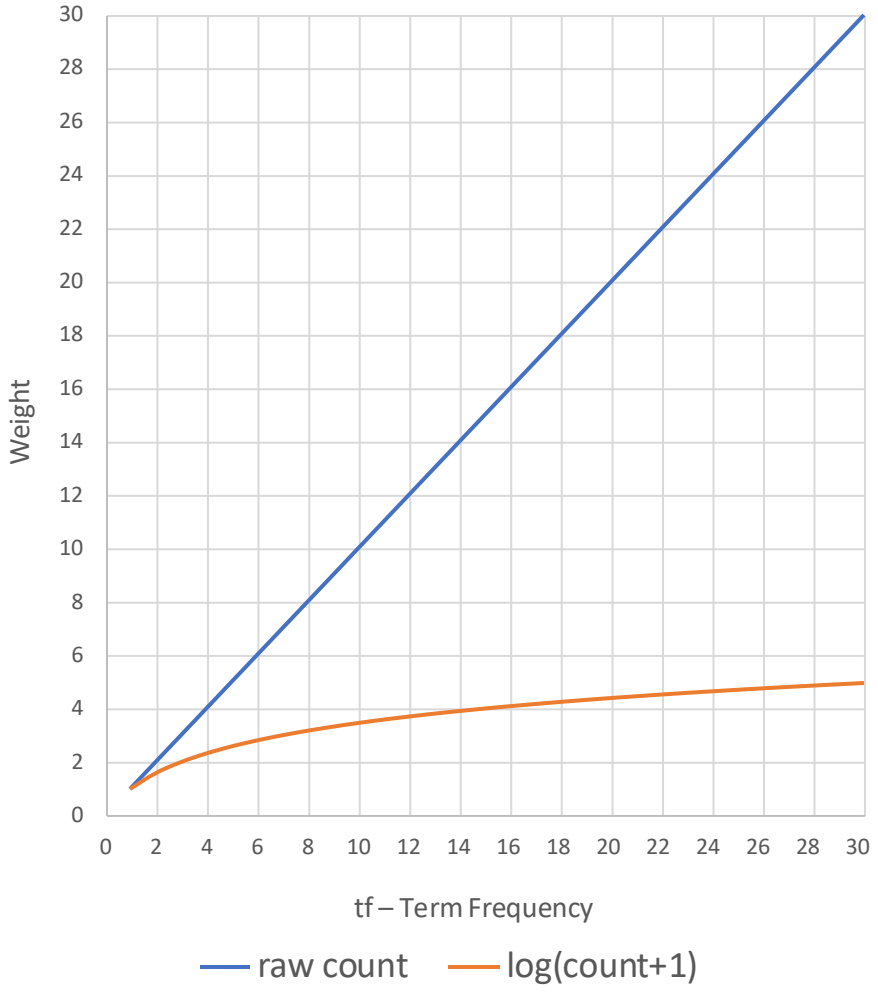
From: A probabilistic model of information retrieval: development and comparative experiments, Spärck Jones et al. http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.134.6108&rep=rep1&type=pdf

Term Frequency – conceptional data view

- **Bag of words**: word order is not important
- First step for a retrieval model: number of occurrences counts!
- $tf_{t,d}$ number of occurrences of term *t* in document *d*

		Documents					
		Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Terms	Antony	157	73	0	0	0	0
	Brutus	4	157	0	1	0	0
	Caesar	231	227	0	2	1	1
	Calpurnia	0	10	0	0	0	0
	Cleopatra	57	0	0	0	0	0
	mercy	2	0	3	5	5	1
	worser	2	0	1	1	1	0

Term Frequency & Logarithm



- In long documents, a term may appear hundred of times.
- Retrieval experiments show that using the logarithm of the number of term occurrences is more effective than raw counts.
- Commonly used approach: apply logarithm

$$\log(1 + tf_{t,d})$$

Document Frequency

- df_t = in how many documents does term t appear in

- Rare terms are more informative than frequent terms
  - Recall function words (and, or, the, …)
- Consider a term in the query that is rare in the collection
  - e.g., *Darmstadt in a news corpora*
- A document containing this term is very likely to be relevant to the query *TU Darmstadt*

→ We want a high weight for rare terms like *Darmstadt*.

IDF – Inverse Document Frequency

- A common way of defining the inverse document frequency of a term is as follows:

$$\text{idf}(t) = log\,\frac{|D|}{df_t}$$

- df_t is an inverse measure of the "informativeness" of the term
- $df_t \leq |D|$
- Logarithm is used also for *idf* to "dampen" its effect.

$\|D\|$	Total # of documents
df_t	# of Documents with $tf_{t,} > 0$

TF-IDF

$$TF_IDF(q,d) = \sum_{t \in T_d \cap T_q} log(1 + tf_{t,d}) * log(\frac{|D|}{df_t})$$

increases with the number of occurrences within a document

increases with the rarity of the term in the collection

- A rare word (in the collection) appearing a lot in one document creates a high score
- Common words are downgraded

For more variations: https://en.wikipedia.org/wiki/Tf-idf

$\sum_{t \in T_d \cap T_q}$ Sum over all query terms, that are in the index

$tf_{t,d}$ Term frequency

$|D|$ Total # of documents

df_t # of Documents with $tf_{t,d} > 0$

TF-IDF – Usage

- Useful not only as a standalone model in document retrieval
- Weights used as a base for many other retrieval models
  - Example: Vector Space Model (VSM) works better with tf-idf weights
- Also useful as a generic word weighting mechanism for NLP
  - Task agnostic importance of a word in a document in a collection
  - Assign every word in a collection its tf-idf score

Example

```
from sklearn.feature_extraction.text import TfidfVectorizer
import torch
from torch.nn.functional import cosine_similarity as sim

if __name__ == '__main__':
    corpus = ['I did not hit her',
              'I did not',
              'Oh hi Mark']

    tfidf = TfidfVectorizer(stop_words='english')
    x = torch.tensor(tfidf.fit_transform(corpus).todense())
    fs = tfidf.get_feature_names_out()
    print(fs)
    print(x)

    print(corpus[0], corpus[1], sim(x[0], x[1], dim=0), sep='\t')
    print(corpus[0], corpus[2], sim(x[0], x[2], dim=0), sep='\t')
    print(corpus[1], corpus[2], sim(x[1], x[2], dim=0), sep='\t')

    print(fs[0], fs[2], sim(x[:,0], x[:,2], dim=0), sep='\t')
    print(fs[1], fs[2], sim(x[:,1], x[:,2], dim=0), sep='\t')
```

What the example computed

- x has one **row per document** and one **column per term** (TF-IDF weights).
- Rows: sim(doc, doc) = relevance between texts.
- Columns: sim(term, term) = relatedness between words, through the documents they share.
- This is Lesk without a dictionary: a word's "gloss" is the set of documents it occurs in.
- Limitation: two words that never share a document get similarity exactly 0.

Pair	cos
"I did not hit her" – "I did not"	0.61
"I did not hit her" – "Oh hi Mark"	0.00
did – hit	0.52
hi – hit	0.00

Output with sklearn's English stop-word list (I, not, her, oh … are removed; features: did, hi, hit, mark).

MAX PLANCK SCHOOL of biomedical artificial intelligence

PhD Positions at the Interface of AI and the Life Sciences

Develop new AI approaches for biological systems in a Germany-wide interdisciplinary PhD program.

3 YEARS
Interdisciplinary PhD program

30+ GROUPS
Leading research groups across Germany

DUAL SUPERVISION
Expertise from different research areas

Research fields

- Molecular & Cellular Systems
- Protein & Molecule Design
- Complex Biological Traits from Multi-Scale Data
- Neural Networks

APPLY NOW

APPLICATION DEADLINE
1 DECEMBER 2026

PHD STARTS
SEPTEMBER 2027

SCAN TO EXPLORE THE PROGRAM
https://biomedicalai.maxplanckschools.org

BM25

- Created 1994 by Robertson et al.
- Grounded in probabilistic retrieval

- In general, BM25 improves on TF-IDF results

- But only set as a default scoring in Lucene in 2015

Original paper: http://www.staff.city.ac.uk/~sb317/papers/robertson_walker_sigir94.pdf

TF-IDF vs BM25 in Lucene https://opensourceconnections.com/blog/2015/10/16/bm25-the-next-generation-of-lucene-relevation/

BM25, one component at a time

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Component	What it does	Where it comes from
log((|D| − df + 0.5) / (df + 0.5))	rare terms count more	probability ranking principle + binary independence model
tf (k_1+1) / (tf + k_1 ·…)	more occurrences help, with diminishing returns	2-Poisson “eliteness” model, approximated
1 − b + b · dl / avgdl	long documents need more occurrences	verbosity hypothesis

Plan: start from the question “what is the probability that d is relevant to q?” and build the formula up step by step.

Robertson & Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond, FnTIR.

Step 1: rank by probability of relevance

$$\text{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Robertson-Spärck Jones (RSJ) weight

Probability ranking principle (Robertson, 1977): rank documents by P(R = 1 | d, q). If these probabilities are estimated well, this ranking is optimal.

$$P(R{=}1 \mid d,q) \overset{\text{rank}}{=} \frac{P(R{=}1 \mid d,q)}{P(R{=}0 \mid d,q)} = \frac{P(d \mid R{=}1,q)}{P(d \mid R{=}0,q)} \cdot \underbrace{\frac{P(R{=}1 \mid q)}{P(R{=}0 \mid q)}}_{\text{same for every } d}$$

Step 2a: a document as a binary vector

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

The query q is fixed for the whole derivation, so every probability is "for this query". From now on we write P(d | R) as shorthand for P(d | R, q).

Represent d only by which terms it contains: $x_t = 1$ if t occurs in d, else 0. This is the set view from TF-IDF; counts are ignored until step 4.

	elephant	weight	zoo	lion
"the elephant at the zoo"	1	0	1	0

Each term t then has two probabilities, one per class:

	meaning	complement
$p_t = P(x_t = 1 \mid R = 1)$	how often t appears in *relevant* documents	$P(x_t = 0 \mid R = 1) = 1 - p_t$
$s_t = P(x_t = 1 \mid R = 0)$	how often t appears in *non-relevant* documents	$P(x_t = 0 \mid R = 0) = 1 - s_t$

A good query term has high p_t and low s_t: for "elephant weight", elephant is in almost every relevant document and in very few others.

Step 2b: independence turns the ratio into a product

$$\text{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Assumption: given the relevance class, terms occur independently of each other (the "naive Bayes" assumption). Clearly false (elephant and trunk co-occur), but it makes the model tractable:

$$\frac{P(d \mid R{=}1)}{P(d \mid R{=}0)} = \prod_{t \in V} \frac{P(x_t \mid R{=}1)}{P(x_t \mid R{=}0)}$$

Each factor takes one of two values, depending on whether t is in d:

$$= \prod_{t:\, x_t = 1} \underbrace{\frac{p_t}{s_t}}_{t \text{ present}} \cdot \prod_{t:\, x_t = 0} \underbrace{\frac{1 - p_t}{1 - s_t}}_{t \text{ absent}}$$

Step 2c: only query terms matter

$$\text{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Assumption: a term that is not in the query is equally likely in relevant and non-relevant documents, $p_t = s_t$. Its factor is $p_t/s_t = 1$ or $(1 - p_t)/(1 - s_t) = 1$, so it disappears:

$$\frac{P(d \mid R{=}1)}{P(d \mid R{=}0)} = \prod_{t \in T_q \cap T_d} \frac{p_t}{s_t} \cdot \prod_{t \in T_q \setminus T_d} \frac{1 - p_t}{1 - s_t}$$

Query terms …	**contribute**	**example: q = “elephant weight”, d = “the elephant at the zoo”**
in the document ($T_q \cap T_d$)	p_t / s_t	elephant
missing from the document ($T_q \setminus T_d$)	$(1 - p_t)$ / $(1 - s_t)$	weight

Problem: the second product still depends on d, since different documents miss different query terms.

Step 2d: move everything onto the matched terms

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Multiply and divide by the "absent" factor of every *matched* term, then regroup:

$$\prod_{t \in T_q \cap T_d} \frac{p_t}{s_t} \prod_{t \in T_q \setminus T_d} \frac{1-p_t}{1-s_t} = \prod_{t \in T_q \cap T_d} \frac{p_t\,(1-s_t)}{s_t\,(1-p_t)} \cdot \underbrace{\prod_{t \in T_q} \frac{1-p_t}{1-s_t}}_{\text{all query terms}}$$

Check, t matched: $\dfrac{p_t\,(1-s_t)}{s_t\,(1-p_t)} \cdot \dfrac{1-p_t}{1-s_t} = \dfrac{p_t}{s_t}$ ✓

Check, t missing: $\dfrac{1-p_t}{1-s_t}$ (only in the last product) ✓

The last product runs over *all* query terms, so it is the same for every document: a positive constant. Dropping it is rank-equivalent.

Step 2e: take logs: the RSJ weight

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

log is strictly increasing (rank-equivalent) and turns the product into a sum:

$$\mathrm{score}(d) = \sum_{t \in T_q \cap T_d} w_t, \qquad w_t = \log \frac{p_t\,(1-s_t)}{s_t\,(1-p_t)} = \underbrace{\log \frac{p_t}{1-p_t}}_{\text{log-odds in relevant}} - \underbrace{\log \frac{s_t}{1-s_t}}_{\text{log-odds in non-relevant}}$$

Term	p_t	s_t	w_t
elephant	0.9	0.01	6.8
the	0.5	0.5	0

Illustrative values (natural log). Robertson & Spärck Jones (1976).

- w_t depends only on the term (for this query), not on d. The document only decides *which* weights get summed.
- Sum over $T_q \cap T_d$: the set intersection of Lesk and TF-IDF, now with a weight derived from probability.

Next: without relevance judgements we cannot count p_t and s_t. Step 3 estimates them.

Step 3a: what can we estimate without labels?

$$\text{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Split the RSJ weight from step 2e into a relevant part and a non-relevant part:

$$w_t = \log \frac{p_t\,(1-s_t)}{s_t\,(1-p_t)} = \underbrace{\log \frac{p_t}{1-p_t}}_{\text{needs relevant documents}} + \underbrace{\log \frac{1-s_t}{s_t}}_{\text{needs non-relevant documents}}$$

Part	**To estimate it we would count …**	**Without relevance judgements**
p_t (relevant side)	how many *relevant* documents contain t	no relevant documents to count → step 3b
s_t (non-relevant side)	how many *non-relevant* documents contain t	almost the whole collection is non-relevant → step 3c

Croft & Harper (1979), Using probabilistic models of document retrieval without relevance information, J. Documentation.

Step 3b: the relevant side vanishes

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Assumption (Croft & Harper, 1979): with no information about relevant documents, a query term is as likely to be present as absent in them, so $p_t = 0.5$ for every query term.

$$\log \frac{p_t}{1 - p_t} \overset{p_t = 0.5}{=} \log \frac{0.5}{0.5} = \log 1 = 0$$

$$\Rightarrow \quad w_t = 0 + \log \frac{1 - s_t}{s_t}$$

Every query term gets the same contribution (zero) from the relevant side, so the weight is decided entirely by how common the term is among non-relevant documents.

Step 3c: the non-relevant side becomes df

$$\text{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Assumption: relevant documents are a tiny fraction of the collection, so the non-relevant documents ≈ all documents. The fraction of them containing t is then:

$$s_t \approx \frac{df_t}{|D|} \qquad 1 - s_t \approx \frac{|D| - df_t}{|D|}$$

Substitute both, and the |D| in the two denominators cancels:

$$\log \frac{1 - s_t}{s_t} \approx \log \frac{(|D| - df_t)\,/\,|D|}{df_t\,/\,|D|} = \log \frac{|D| - df_t}{df_t} \overset{df_t \ll |D|}{\approx} \log \frac{|D|}{df_t}$$

For rare terms this is exactly the IDF of TF-IDF. Remaining question: where do the +0.5 in BM25 come from?

Step 3d: the +0.5 come from smoothed counts

$$\text{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

With R judged relevant documents, r_t of which contain t:

	relevant	non-relevant	total
contain t	r_t	$df_t - r_t$	df_t
do not	$R - r_t$	$\|D\| - df_t - R + r_t$	$\|D\| - df_t$
total	R	$\|D\| - R$	$\|D\|$

Estimate p_t and s_t with 0.5 added to every count (avoids 0 and log 0):

$$p_t = \frac{r_t + 0.5}{R + 1} \qquad s_t = \frac{df_t - r_t + 0.5}{|D| - R + 1}$$

No relevance judgements: set $R = r_t = 0$. Both assumptions from 3b and 3c fall out, and the RSJ weight becomes BM25's IDF:

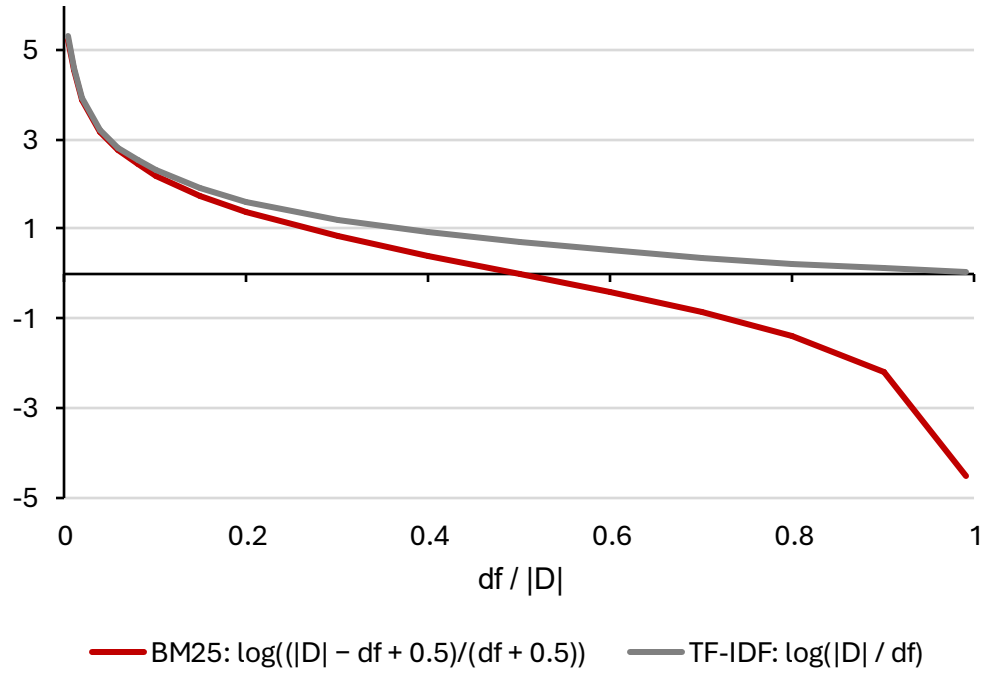
$$p_t = \frac{0.5}{1} = 0.5, \quad s_t = \frac{df_t + 0.5}{|D| + 1} \approx \frac{df_t}{|D|} \quad \Rightarrow \quad w_t = \log \frac{0.5\,(|D| - df_t + 0.5)}{(df_t + 0.5)\,0.5} = \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Robertson & Spärck Jones (1976), Relevance weighting of search terms, JASIS; Robertson & Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond, FnTIR.

Step 3e: the result, compared with TF-IDF

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

- Rare terms: |D| − df ≈ |D|, so it matches the TF-IDF weight log(|D|/df).
- Terms in more than half of the documents get a *negative* weight: they are more typical of non-relevant documents. Implementations often clip at 0 or use log(1 + …) (e.g. Lucene).



Robertson & Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond, FnTIR. §3.

Step 4a: why counts should matter, but not linearly

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

So far a term is either present or absent. Query “elephant weight”:

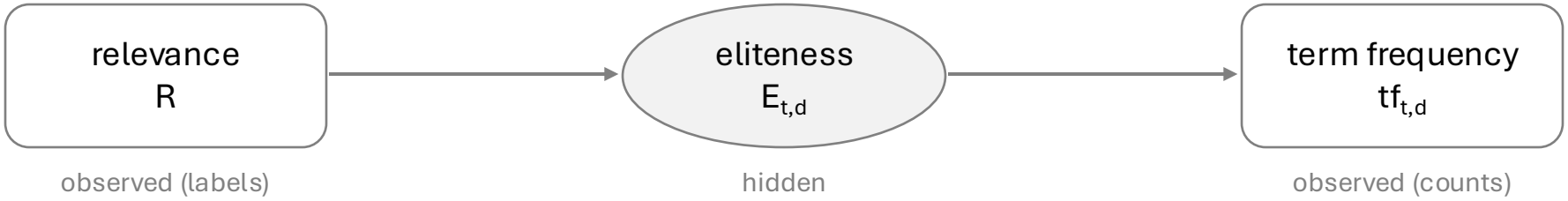
Document	tf(elephant)	Binary model	Intuition
an article about elephants	20	present	clearly about elephants
a zoo guide with one line on elephants	1	present	mentions them in passing
the same article, twice as long, with the same content	40	present	not twice as relevant

- We want more occurrences to count as more evidence, with diminishing returns.
- Problem: relevance is about what a document is *about*, while tf is a surface count. We need a model that connects the two. Step 4b introduces it.

Step 4b: eliteness, a hidden “aboutness” variable

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

For every term–document pair, introduce a hidden binary variable $E_{t,d}$: is d *about* the concept behind t (elite) or not? (Harter, 1975)



- **Relevance → eliteness**: relevant documents are more likely to be elite for the query terms. π_R = P(elite | R).
- **Eliteness → tf**: tf depends only on eliteness. Once we know whether d is about elephants, relevance tells us nothing more about how often “elephant” appears.

Step 4c: the 2-Poisson model

$$\text{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

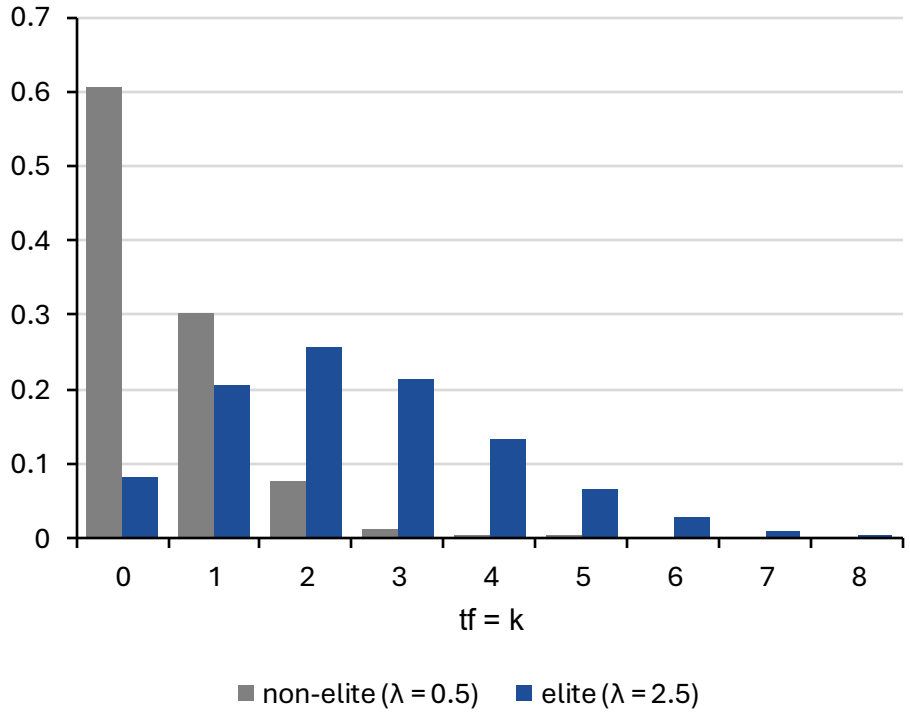
Within one eliteness class, treat occurrences of t as random events at a constant rate, giving a Poisson distribution with one rate per class:

$$P(tf = k \mid E) = \frac{e^{-\lambda_E}\,\lambda_E^k}{k!}, \qquad \lambda_{\text{elite}} > \lambda_{\text{non}}$$

Relevance only changes the mix of the two classes:

$$P(tf = k \mid R) = \pi_R\,\text{Pois}(k;\lambda_{\text{elite}}) + (1-\pi_R)\,\text{Pois}(k;\lambda_{\text{non}})$$

Harter (1975): content words follow such a mixture of two Poissons; function words (“the”, “of”) fit a single Poisson.



Harter (1975), A probabilistic approach to automatic keyword indexing, JASIS. Rates in the chart are illustrative.

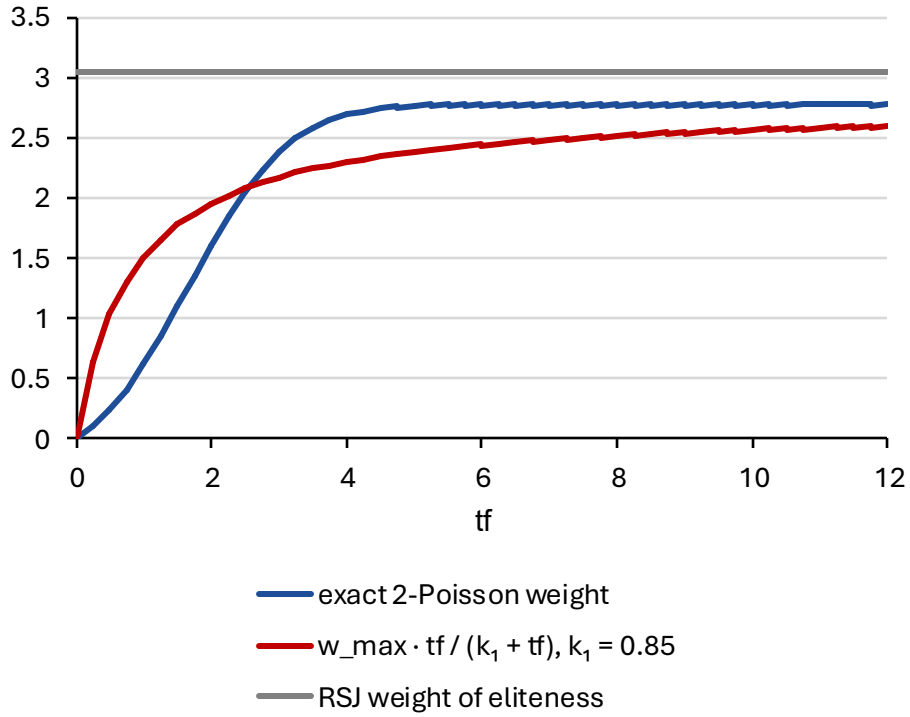
Step 4d: the weight this model implies

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Same log-odds idea as step 2, but comparing "tf = k" with "tf = 0" instead of present with absent:

$$w(k) = \log \frac{P(tf{=}k \mid R{=}1)\; P(tf{=}0 \mid R{=}0)}{P(tf{=}k \mid R{=}0)\; P(tf{=}0 \mid R{=}1)}$$

- Its shape (Robertson & Walker, 1994): w(0) = 0, it increases with tf, and it levels off at a maximum close to the RSJ weight of eliteness itself.
- But each term needs four unknown parameters (two rates, two mixing weights), which we cannot estimate without relevance data.
- Keep the key properties (0 at tf = 0, increasing, saturating), not the exact curve: step 4e.



Illustrative parameters: λ = 0.5 / 2.5, π = 0.7 (relevant) / 0.1 (non-relevant). Robertson & Walker (1994), SIGIR.

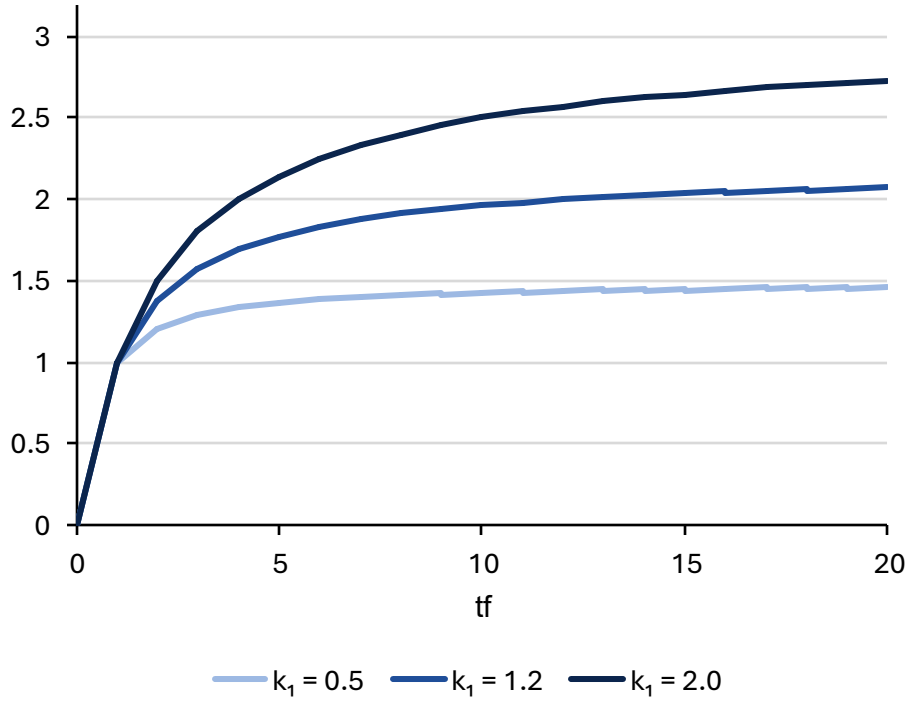
Step 4e: one parameter for the shape: k_1

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

A simple function with the same three properties (0 at tf = 0, increasing, saturating):

$$\frac{tf}{k_1 + tf} \xrightarrow{\times\,(k_1+1)} \frac{tf\,(k_1+1)}{k_1 + tf}$$

k_1	tf factor	behaves like
0	1 for any tf > 0	binary model (step 2)
1.2 – 2.0	saturating; half the cap at tf = k_1	typical BM25
→ ∞	→ tf	raw term frequency



×(k_1 + 1) is a constant (rank-equivalent): tf = 1 scores 1 and the cap is k_1 + 1. A saturated term earns (k_1 + 1) × its IDF.

Robertson & Walker (1994), Some simple effective approximations to the 2-Poisson model, SIGIR.

Step 5: document length

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

- **Verbosity**: a long document may say the same thing with more words, so its tf is inflated.
- **Scope**: a long document may cover more topics, so its tf is genuine. b interpolates between the two views.

$$B = (1-b) + b\,\frac{dl_d}{avgdl}, \qquad \widetilde{tf} = \frac{tf}{B} \quad \Rightarrow \quad \frac{\widetilde{tf}\,(k_1+1)}{\widetilde{tf} + k_1} = \frac{tf\,(k_1+1)}{tf + k_1 B}$$

Setting	B	Effect
b = 0	1	no length normalization
dl = avgdl	1	an average-length document is left unchanged, for any b
dl = 2·avgdl, b = 0.75	1.75	tf = 4 counts like tf ≈ 2.3

Putting it together: one term, one document

$$\mathrm{BM25}(q,d) = \sum_{t \in T_q \cap T_d} \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,\frac{dl_d}{avgdl}\right)} \cdot \log \frac{|D| - df_t + 0.5}{df_t + 0.5}$$

Quantity	Value
$\lvert D\rvert$, df_t	1,000, 10
$tf_{t,d}$	3
dl_d, avgdl	200, 100
k_1, b	1.2, 0.75

$\log \frac{1000-10+0.5}{10+0.5} = 4.55$ rare term

$B = 0.25 + 0.75 \cdot 2 = 1.75$ twice the average length

$\frac{3 \cdot 2.2}{3 + 1.2 \cdot 1.75} = 1.29$ below the cap of 2.2

$\mathrm{BM25} = 1.29 \cdot 4.55 = 5.88$

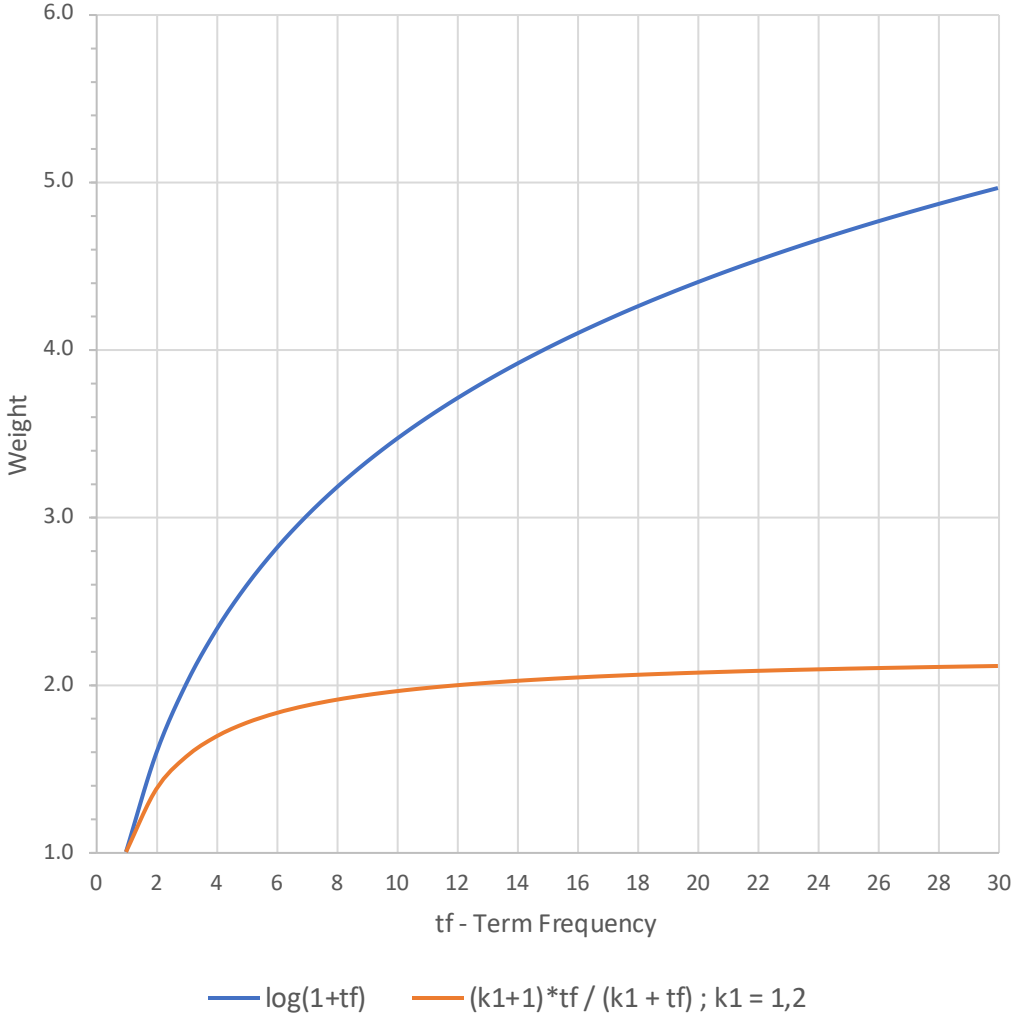
Sum this over every query term that occurs in d. The full model also has a query-tf factor (k_3), usually dropped for short queries.

Details: Robertson & Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond, FnTIR.

BM25 vs. TF-IDF

- Simple case of BM25 looks a lot like TF-IDF
- 1 main difference: BM25 tf component contains saturation function
  - Therefore works better in practice
- BM25 variants can be adapted to:
  - Incorporate additional reference information
  - Long(er) queries
  - multiple fields

BM25 vs. TF-IDF - Saturation



- **TF-IDF:** weight is always increasing (even with log)
- **BM25:** diminishing returns quickly = asymptotically approaches $k_1 + 1$

Note: we added (k_1+1) to the numerator to make tf@1 = 1, but it does not change the ranking because it is added to every term

Note: we assume the doc length = avgdl

BM25 vs. TF-IDF - Example

- Suppose your query is “machine learning”
- Suppose you have 2 documents with term counts:
  - doc1: learning 1024; machine 1
  - doc2: learning 16; machine 8

- TF-IDF: $\log(\text{tf}) * \log(|D|/\text{df})$
  - doc1: $11 * 7 + 1 * 10 = \mathbf{87}$
  - doc2: $5 * 7 + 4 * 10 = \mathbf{75}$
- BM25: $k_1 = 2$
  - doc1: $3 * 7 + 1 * 10 = \mathbf{31}$
  - doc2: $2.67 * 7 + 2.4 * 10 = \mathbf{42.7}$

Example taken from: https://web.stanford.edu/class/cs276/

Hyperparameters

- k_1, b are hyperparameters = they are set by the developers

- k_1 controls term frequency scaling
  - k_1 = 0 is binary model; k_1 large is raw term frequency
- b controls document length normalization
  - b = 0 is no length normalization; b = 1 is relative frequency (fully scale by document length)

- Common ranges: 0.5 < b < 0.8 and 1.2 < k_1 < 2

Summary: retrieval with sets

1. We save statistics about terms in an inverted index

2. The statistics in the index can be accessed by a given term (query)

3. TF-IDF & BM25 use term and document frequencies to score a query & doc

Where sets fail

- TF-IDF is actually pretty naive:
  - Car vs cars → lemmatisation, wordnet…
  - Car vs automobile → different tokens!
  - No generalisation

- TF-IDF is **very sparse**:
  - We need to keep track of an **M x N** matrix of token frequencies
    - M: documents, N: vocab size
  - Say 10,000 words, 10,000 documents: 100M values
  - BERT & GPT2: 410M parameters

Next lecture: geometry

Structure imposed	Context of a word	A word is …	Relatedness is …	Methods
Sets	the documents it is a member of	an atomic symbol (an index)	counted shared membership	Lesk, TF-IDF, BM25
Geometry	a document or a ±k window (order ignored)	a point in $\mathbb{R}^k$	angle / distance, linear offsets	LSA, word2vec, GloVe
Positions	the ordered sequence around it	a vector that depends on its context	probability of a word at a position	neural LMs (RNN → Transformer)

Sets cannot relate car and automobile unless they co-occur. Geometry lets words without a shared document still be close.

Next: LSA → word2vec → GloVe, how they are related, and how to evaluate representations (including retrieval).